

Complete Explanation — Step by Step

Button color scheme

Each button type uses a distinct color so users can find keys at a glance:

Digit buttons (0–9)	Indigo — neutral, used most often
Operator buttons ($\div$ $\times$ $-$ $+$)	Violet — math operations
Function buttons ($+/-$ $\%$ $.$)	Teal — secondary utilities
Clear button (AC)	Red — destructive / reset
Equals button ($=$)	Green — confirm / result

HTML Structure

Step 1

Reset & page centering

Every browser applies its own default margins and padding to elements. The universal selector (*) strips all of them so the layout starts from a clean base. `box-sizing: border-box` ensures padding is included inside element dimensions — no accidental overflow. The body uses flexbox to center the calculator perfectly in the viewport.

```
* { box-sizing: border-box; margin: 0; padding: 0; }
body {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  background: #0f0f1a;
}
```

Step 2**Calculator shell (.calc)**

The outer container is a dark rounded rectangle that holds the display and button grid. `flex-direction: column` stacks children vertically. `gap: 10px` adds breathing room between the display and grid without manual margins.

```
.calc {  
  width: 300px;  
  background: #1e1e2e; /* dark purple */  
  border-radius: 20px;  
  padding: 1.25rem;  
  display: flex;  
  flex-direction: column;  
  gap: 10px;  
}
```

Step 3**Display screen (.display, .expr, .num)**

The display has two text layers. The top line (`.expr`) shows the running expression like '8 x 5 =' in a small muted font. The bottom line (`.num`) shows the current number in a large bright font. `align-items: flex-end` right-aligns both lines, exactly like a physical calculator.

```
.display {  
  background: #13131f;  
  border-radius: 14px;  
  padding: 1rem 1.25rem;  
  min-height: 96px;  
  display: flex;  
  flex-direction: column;  
  align-items: flex-end; /* right-align */  
  justify-content: flex-end; /* stick to bottom */  
}  
.expr { font-size: 13px; color: #6c6c8a; } /* muted */  
.num { font-size: 38px; color: #e0e0ff; } /* bright */
```

Step 4**Button grid (.grid)**

CSS Grid creates the 4-column layout automatically. `repeat(4, 1fr)` means four equal-width columns. `gap: 8px` spaces every button without any margin math.

```
.grid {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  gap: 8px;  
}
```

Step 5 Base button (.btn) + press effect

All buttons share common rules: fixed height, no border, rounded corners, pointer cursor. The :active rule fires when the button is held down, scaling it to 92% to create a satisfying tactile press feeling.

```
.btn {
  height: 60px;
  border: none;
  border-radius: 14px;
  font-size: 19px;
  font-weight: 500;
  cursor: pointer;
  transition: transform 0.08s, filter 0.1s;
}
.btn:active {
  transform: scale(0.92); /* shrinks on press */
  filter: brightness(1.15); /* brightens on press */
}
```

Step 6 Color roles for each button type

Five color classes give each button group a distinct identity. Users learn the layout by color, reducing mistakes.

```
.btn.num-btn { background: #2e2e4e; color: #c8c8f0; } /* indigo */
.btn.op { background: #7c3aed; color: #f0e8ff; } /* violet */
.btn.fn { background: #0e7490; color: #e0f7fa; } /* teal */
.btn.clr { background: #be123c; color: #ffe4ec; } /* red */
.btn.eq { background: #16a34a; color: #e8fff0; } /* green */
```

Step 7 Wide zero button

The '0' key on a classic calculator spans two columns. grid-column: span 2 achieves this in one line — no extra wrappers needed.

```
.btn.zero {
  grid-column: span 2;
}
```

JavaScript Logic

Step 8**State variables (the calculator's memory)**

Four variables track everything the calculator knows at any moment. They are declared inside an IIFE (Immediately Invoked Function Expression) so they don't pollute the global window object.

- `cur` — whatever is shown on screen (stored as a string to preserve '0.' etc.)
- `prev` — the first number, saved when the user presses an operator
- `op` — which operator was pressed last (null if none)
- `fresh` — when true, the next digit replaces the display instead of appending

```
(function () {  
  let cur = '0'; // number on screen right now  
  let prev = ''; // number saved before operator  
  let op = null; // pending operator: + - x /  
  let fresh = false; // next digit starts a new number  
  
  const D = document.getElementById('disp');  
  const E = document.getElementById('expr');  
})();
```

Step 9**Number formatter: `fmt()`**

JavaScript floating-point math has a famous quirk: $0.1 + 0.2 = 0.30000000000000004$. `fmt()` uses `toPrecision(12)` to round away these artifacts before showing the result.

```
function fmt(n) {  
  const s = parseFloat(n.toPrecision(12)).toString();  
  return s.length > 13  
    ? parseFloat(n.toPrecision(9)).toString()  
    : s;  
}
```

**Step
10****Compute result: calc()**

When the user presses '=' (or chains operators), `calc()` reads `prev` and `cur`, applies `op`, and writes the result back to `cur`. Division by zero returns the string 'Error' instead of Infinity.

```
function calc() {
  const a = parseFloat(prev), b = parseFloat(cur);
  if (isNaN(a) || isNaN(b)) return;

  let r;
  if (op === '/') { r = b === 0 ? 'Error' : a / b; }
  else if (op === 'x') { r = a * b; }
  else if (op === '-') { r = a - b; }
  else { r = a + b; }

  E.textContent = prev + ' ' + op + ' ' + cur + ' =';
  cur = r === 'Error' ? 'Error' : fmt(r);
  prev = ''; op = null; fresh = true;
  show();
}
```

**Step
11****Append a digit: digit()**

`digit()` appends a character to `cur` safely. It handles three edge cases: replacing a leading zero, capping input at 15 characters, and starting fresh after a result.

```
function digit(v) {
  if (cur === 'Error') cur = '0';
  if (fresh) { cur = v; fresh = false; }
  else if (cur === '0' && v !== '.') { cur = v; } // replace zero
  else if (cur.length < 15) { cur += v; } // cap at 15
  show();
}
```

**Step
12**

Set an operator: setOp()

When the user presses +, -, x, or /, setOp() saves the current number into prev and records the operator. If an operator is already pending and the user hasn't typed a new number, it first evaluates the existing expression — this enables chained calculations like 3 + 5 x 2.

```
function setOp(o) {
  if (op && !fresh) calc(); // evaluate chain first
  if (cur === 'Error') return;
  prev = cur; op = o; fresh = true;
  E.textContent = prev + ' ' + op;
}
```

**Step
13**

Wire up button clicks

A single querySelectorAll loop attaches one listener to every button. The data-a attribute on each button tells the handler what to do, keeping the HTML and JS loosely coupled.

```
document.querySelectorAll('.btn').forEach(b => {
  b.addEventListener('click', () => {
    const a = b.dataset.a;
    if (a === 'd') digit(b.dataset.v);
    else if (a === 'op') setOp(b.dataset.o);
    else if (a === 'eq') { if (op) calc(); }
    else if (a === 'clear') { cur=''; prev=''; op=null;
      fresh=false; E.textContent=''; show(); }
    else if (a === 'sign') { cur = cur.startsWith('-')
      ? cur.slice(1) : '-' + cur; show(); }
    else if (a === 'pct') { cur = fmt(parseFloat(cur)/100); show(); }
    else if (a === 'dot') { if (!cur.includes('.')) cur+='.'; show(); }
  });
});
```

**Step
14****Keyboard support**

A keydown listener on document maps physical keys to the same functions used by buttons. Backspace deletes the last digit; Escape clears everything; / is prevented from triggering the browser's Find dialog.

```
document.addEventListener('keydown', e => {
  if (e.key >= '0' && e.key <= '9') digit(e.key);
  else if (e.key === '.') digit('.');
  else if (e.key === '+') setOp('+');
  else if (e.key === '-') setOp('-');
  else if (e.key === '*') setOp('*');
  else if (e.key === '/') {
    e.preventDefault(); // stop browser Find
    setOp('/');
  }
  else if (e.key === 'Enter' || e.key === '=') { if (op) calc(); }
  else if (e.key === 'Backspace') {
    if (!fresh && cur !== 'Error') {
      cur = cur.length > 1 ? cur.slice(0,-1) : '0';
      show();
    }
  }
  else if (e.key === 'Escape') {
    cur='0'; prev=''; op=null; fresh=false;
    E.textContent=''; show();
  }
});
```

Quick reference summary

Step	Section	Key concept
1	Reset & centering	box-sizing + flexbox centering
2	Calculator shell	Dark rounded container with flex column
3	Display screen	Two text lines, right-aligned
4	Button grid	CSS Grid with repeat(4, 1fr)
5	Base button	Height + border-radius + :active scale
6	Color roles	5 color classes encode button type
7	Wide zero	grid-column: span 2
8	State variables	cur, prev, op, fresh inside IIFE
9	Number formatter	toPrecision(12) kills float errors

Step	Section	Key concept
10	calc()	Arithmetic + div-by-zero guard
11	digit()	Append digits with edge-case handling
12	setOp()	Save operand + enable chained ops
13	Button click wiring	data-a attribute as action router
14	Keyboard support	keydown maps physical keys to functions

Built with HTML, CSS, and JavaScript — no external libraries required.